

Programmeringsparadigmer 2025

Kursusgang 6 : Højere-ordens-funktioner

Hans Hyttel

15. oktober 2025

Vores plan for i dag

1. Læringsmål
2. Forberedelsesopgaverne
3. Diskussion – [dbs](#)
4. Opgave 1 : [dbs](#)
5. Opgave 2 : [within](#)
6. Pause
7. Opgave 3 : [sumrows](#)
8. Opgave 4 : Det er helt naturligt
9. Opgave 5 : [fingo](#)
10. Hvis tiden tillader det: Flere opgaver i eget tempo.
11. Evaluering.

Læringsmål

- ▶ At kunne forklare præcist, hvad en højere-ordens-funktion er, og hvad typen af en højere-ordens-funktion er
- ▶ At kunne forklare højere-ordens-funktioner på lister, herunder map, filter, foldr og foldl
- ▶ At kunne forklare og forstå de rekursive definitioner af almindelige højere-ordens-funktioner på lister
- ▶ At kunne anvende højere-ordens-funktioner, herunder map, filter, foldr og foldl, til at løse programmeringsopgaver i Haskell
- ▶ At kunne forklare og bruge funktionssammensætning i Haskell

Forberedelsesopgave – positions

Hvert bogstav i det engelske alfabet (små bogstaver) har en position. "a" har position 1, "c" har position 3, og "h" har position 8. I Haskell er enhver streng en liste af tegn. Så String er den samme type som [Char].

Vi kan definere en funktion `positions`, som, givet en streng af små bogstaver `str`, giver os listen af positioner for tegnene i `str`.

Som eksempel giver `positions "abba"` os `[1,2,2,1]`. Brug højere-ordens-funktioner fra kapitel 7 til at definere `positions`.

Forberedelsesopgave – sumsq

Funktionen `sumsq` tager et heltal n som argument og returnerer summen af kvadraterne af de første n heltal. Så `sumsq n` returnerer summen

$$1 + \dots + n^2$$

Som eksempel giver `sumsq 4` os `30`, og `sumsq 9` giver os `285`.
Brug foldr til at definere `sumsq` – og brug ikke map.

Diskussion – dbs

En præsident for et stort land kan lide at omtale sig selv som ”den største programmør, der nogensinde har levet”. For at imponere en journalist definerede han en funktion `dbs` over lister ved hjælp af højere-ordens-funktioner. Listen skal tage en liste af talpar og returnere listen af tupler, hvor det andet element er det dobbelte af det første element. Som eksempel skal `dbs [(1,4) ,(3,6) ,(9,2) ,(4,8) ]` returnere `[(3,6) ,(4,8) ]`.

Her er, hvad præsidenten skrev:

```
dbs xs | length xs == 0 = []  
      | otherwise = filter ((2 * (fst (head  
                             xs))) == (snd (head xs))) xs
```

Journalisten bemærkede, at der var noget, der ikke var helt rigtigt, og at kodestilen var ret klodset. **Kommentarer, tak!** Definitionen kan findes på Moodle i filen `dbs.hs`.

Opgave 1 – dbs (10 minutter)

Nu er det din tur! Skriv en fornuftig og ikke-klodset definition af **dbs**. Brug højere-ordens-funktioner. Giv gode eksempler på testcases.

Pause

Opgave 2 – within (15 minutter)

Funktionen `within` tager en liste af tal og et par af tal, som repræsenterer et interval, og giver os en liste af tal, der er i inputlisten og inden for intervallet (inklusive endepunkterne) givet af inputparret.

Elementerne i outputlisten skal fremstå i den samme rækkefølge, som de optrådte i inputlisten. Hvis inputparret er `(n1,n2)`, antages det, at `n1` er den nedre grænse for intervallet, og `n2` er den øvre grænse.

Som eksempel skal `within [1,3,4,5,2] (1,3)` give os `[1,3,2]`, og `within [1,3,4,5,2] (3,1)` skal give os `[]`.

Definer `within` ved hjælp af højere-ordens-funktioner fra kapitel 7. Giv gode eksempler på testcases.

Opgave 3 – sumrows (10 minutter)

Implementer funktionen `sumrows`. Funktionen tager en liste af lister af tal og returnerer en endimensional liste af tal, hvor hvert tal er lig med summen af den tilsvarende række i inputlisten. Hvis en liste er tom, er dens sum 0.

Som eksempel skal `sumrows [[1,2], [3,4]]` give os `[3, 7]`, og `sumrows [[],[],[1]]` skal give os `[0,0,1]`.

Definer `sumrows` ved hjælp af højere-ordens-funktioner fra kapitel 7. Giv gode eksempler på testcases.

Opgave 4 – Det er helt naturligt (15 minutter)

Grundtallet for den naturlige eksponentialfunktion $e = 2,718\dots$ kan skrives som grænsen for den uendelige række

$$\sum_{k=0}^{\infty} \frac{1}{k!}$$

Funktionen `approx` skal give os en approksimation af e , som vi finder ved at lægge de første n led af denne uendelige række sammen. Det vil sige at

$$\text{approx } n = \sum_{k=0}^n \frac{1}{k!}$$

Definer `approx` ved hjælp af højere-ordens-funktioner fra kapitel 7; fakultetsfunktionen $k!$ er defineret ved

```
fact k = product [1..k]
```

Giv gode eksempler på testcases.

Opgave 5 – fingo (10 minutter)

Her er en funktion. Hvad gør den? Og hvorfor? Brug forklaringen helt til sidst i afsnit 7.3 i bogen til at hjælpe dig med at besvare ”hvorfor-spørgsmålet.

```
fingo :: [a] -> [a] -> [a]
fingo xs ys = foldr (:) xs ys
```

Evaluering

- ▶ Hvad syntes I var svært?
- ▶ Hvad overraskede jer?
- ▶ Hvad gik godt?